

โครงการรายวิชา Machine Learning

Hotel Booking Cancellation Prediction

ทำนายความเสี่ยงที่การจองโรงแรมจะถูกยกเลิก ด้วยข้อมูลการจองจริง 119,208 รายการ

นายชัยพร ศรีพราย 66114540168

นางสาวกวิณชญา มีศรี 67114540015

นายธนากร ไชยรัตน์ 67114540244

นายรัชชานนท์ รื่นจิตต์ 67114540446

นายราชภัฏ บุญรอด 67114540455

18

features ที่ใช้เทรน

0.847

F1-score บนชุดทดสอบ

ยกเลิก 37% คือรายได้ที่หายและพยากรณ์ห้องที่ผิดพลาด

- ยกเลิกกระชั้นชิด → ขายห้องต่อไม่ทัน → รายได้หายทันที
- โรงแรมต้องทำ Overbooking ขาดขาย ซึ่งถ้าประเมินผิดจะเสียทั้งเงินและชื่อเสียง
- **Input** = ข้อมูลการจอง ณ วันที่จอง → **Output** = ยกเลิก (1) / ไม่ยกเลิก (0)
- ขอบเขต: โรงแรม 2 แห่งในโปรตุเกส ก.ค. 2015 – ส.ค. 2017

37%

ของการจองทั้งหมดถูกยกเลิก
(44,224 จาก 119,390 รายการ)

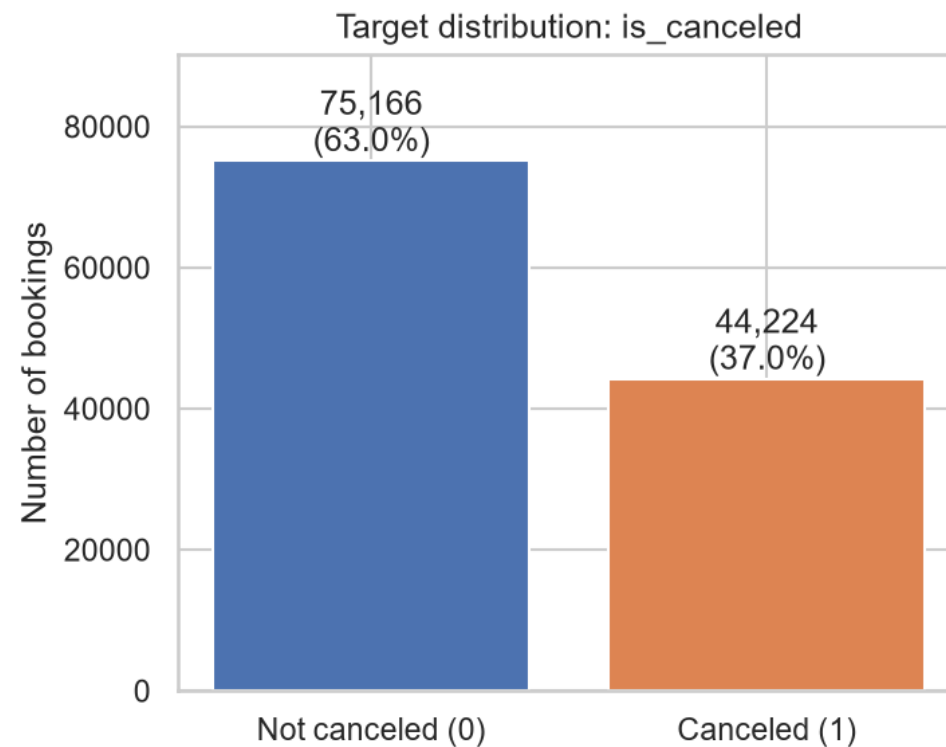
ปัญหานี้เป็น Binary Classification ที่มีมูลค่าทางธุรกิจวัดได้จริง

ทายว่า "ไม่ยกเลิก" ทุกรายการ ก็ได้ Accuracy 62.9% แล้ว

- ข้อมูลดิบ 119,390 แถว × 32 คอลัมน์
- ยกเลิก 44,224 รายการ : ไม่ยกเลิก 75,166 รายการ
- สัดส่วนคลาสไม่สมดุล 37 : 63

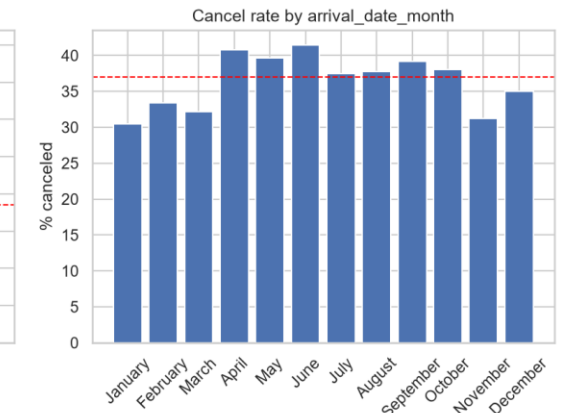
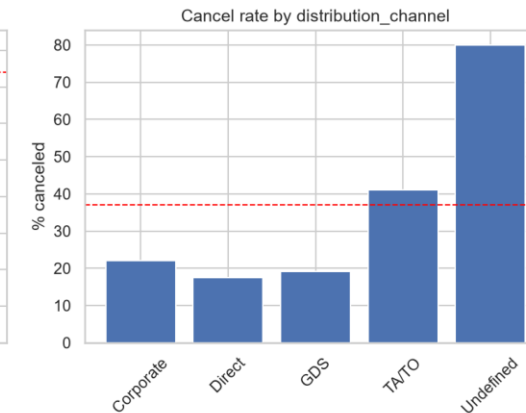
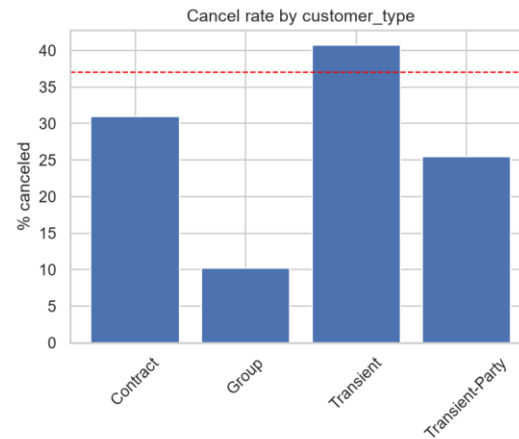
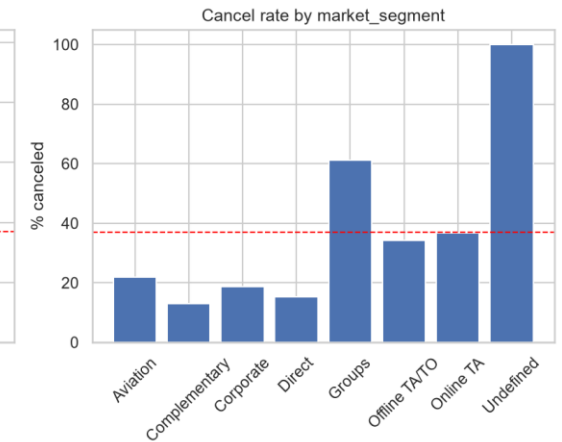
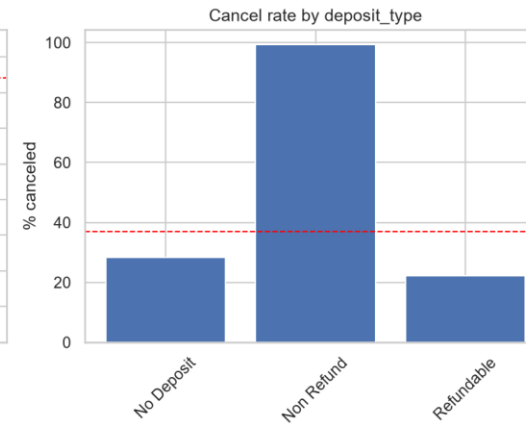
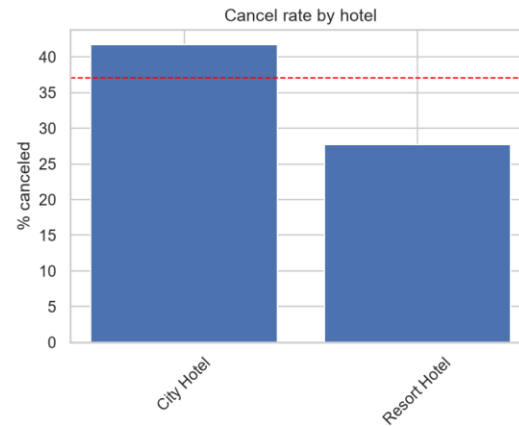
ประโยคสำคัญ:

"นี่คือเหตุผลที่เราไม่ใช้ Accuracy เป็นตัวชี้วัดหลัก เพราะโมเดลที่ไม่ได้เรียนรู้อะไรเลยก็ทำคะแนนนี้ได้"



ประเภทมัดจำและช่องทางการจองต่างกันอย่างชัดเจน

- **Non Refund** ยกเลิกเกือบ 100% ทั้งที่จ่ายเงินไปแล้ว
- **Groups** ยกเลิกสูงกว่าการจองแบบอื่นชัดเจน
- เส้นประแดง = อัตรายกเลิกเฉลี่ยทั้งหมด



ทำความสะอาดข้อมูลโดยเสียข้อมูลเพียง 0.15%

ปัญหาที่พบ	วิธีการจัดการ	แถวที่เสีย
children ว่าง 4 ค่า	เติมด้วย 0	0
country ว่าง 488 ค่า	แทนด้วย Unknown	0
agent / company ว่าง	แปลงเป็น flag มี/ไม่มี	0
ไม่มีผู้เข้าพัก	ลบแถวทิ้ง	180
adr ติดลบ / เกิน 1,000	ลบแถวทิ้ง	2

ค่าว่างใน agent / company ตามเปเปอร์ต้นฉบับแปลว่า “ไม่เกี่ยวข้อง” ไม่ใช่ข้อมูลหาย จึงทำเป็น flag แทนการเติมค่า

119,390 → 119,208

จำนวนแถวก่อน → หลังคลีน

182 แถว

ลบทิ้งทั้งหมด คิดเป็น 0.15%

อัตรายกเลิกหลังคลีน 37.08%
แทบไม่ต่างจาก 37.04% ก่อนคลีน
→ การลบแถวไม่ทำให้สัดส่วนคลาสบิดเบือน

ตัด 3 คอลัมน์ที่แอบรู้คำตอบ ก่อนแบ่งข้อมูล

คอลัมน์ที่ตัดออก

- reservation_status — บอกผลลัพธ์ตรงๆ
- reservation_status_date — วันที่ผลลัพธ์เกิดขึ้น
- assigned_room_type — ห้องถูกกำหนดตอนเช็คอิน

หลักฐาน: การจองที่ถูกเปลี่ยนห้องมีอัตรายกเลิก 0% → รู้ผลหลังเช็คอินแล้ว

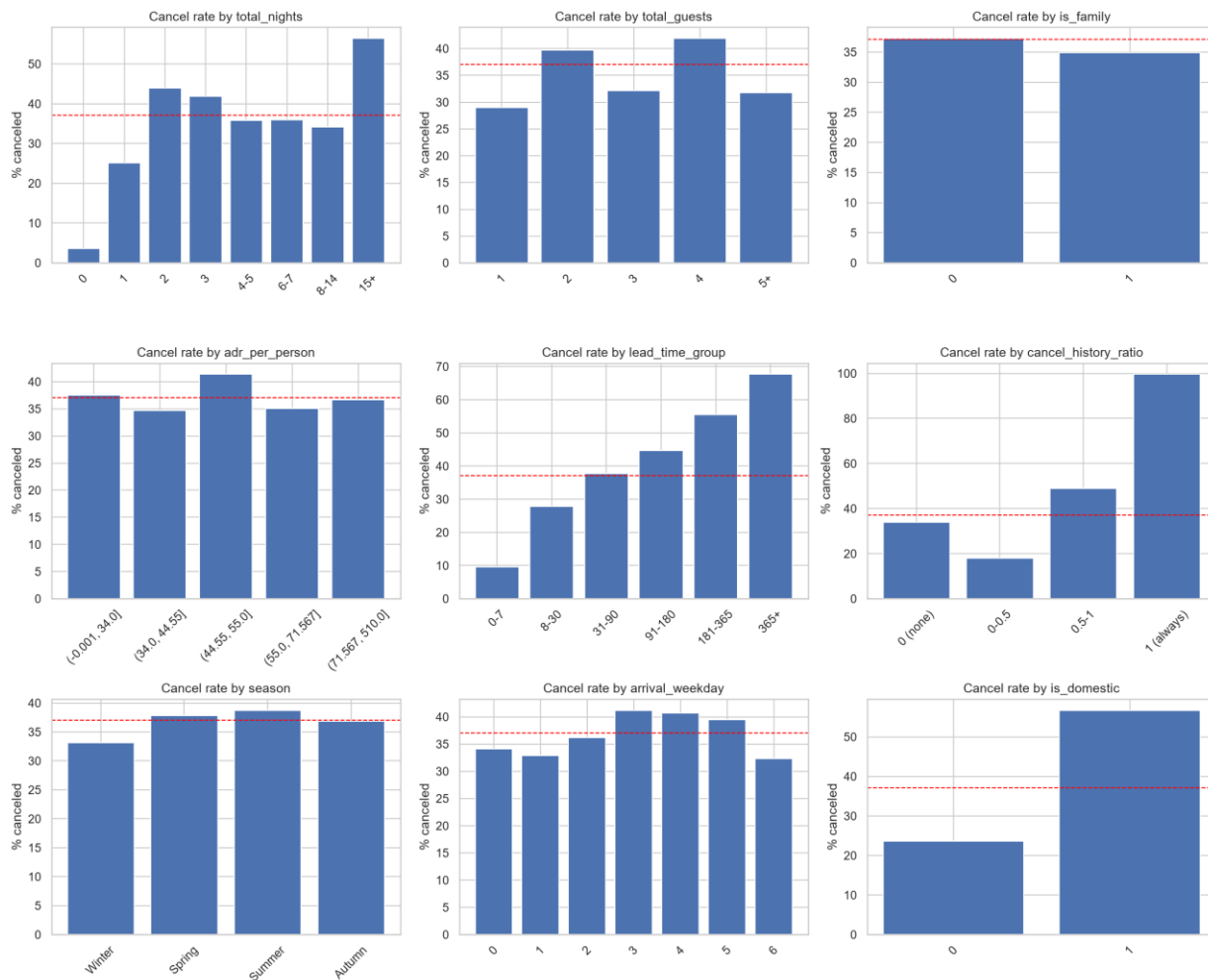
แบ่งข้อมูล 2 แบบ

วิธีแบ่ง	Train	Test
Stratified Random 80/20	95,366	23,842
Time-based (ตามเวลา)	95,190	24,018

Time split: train มีอัตรายกเลิก 36.09% แต่ test มี 40.99%
→ พฤติกรรมเปลี่ยนตามเวลาจริง (ดูหน้า 14)

encoding · scaling · feature selection ทั้งหมดอยู่ใน Pipeline จึงเรียนรู้จาก train ของแต่ละ fold เท่านั้น — Test set ถูกเปิดใช้ครั้งเดียวตอนท้าย

สร้าง 9 features ใหม่ — ทุกตัวมีสมมติฐานรองรับ



ช่วงห่างของอัตรายกเลิกระหว่างกลุ่ม

Feature ใหม่	ช่วงห่าง
cancel_history_ratio	81.6%
lead_time_group	58.1%
total_nights	52.8%
is_domestic	33.1%

ยิ่งช่วงห่างกว้าง feature ยิ่งแยกกลุ่มคนที่จะยกเลิกออกจากคนที่ไม่ยกเลิกได้ดี

ตั้งสมมติฐานก่อน แล้วค่อยพิสูจน์ด้วยกราฟ ไม่ได้สร้างมาแล้วลองดู

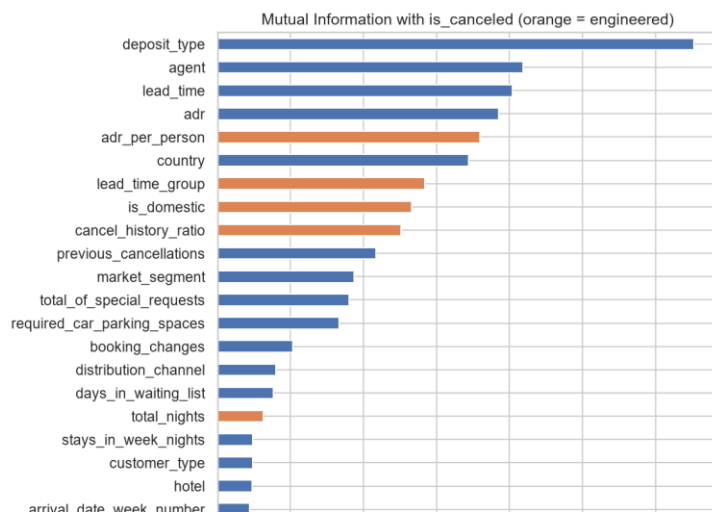
8

FEATURE SELECTION

37 features → 18 features ด้วยการโหวต 2 ใน 3

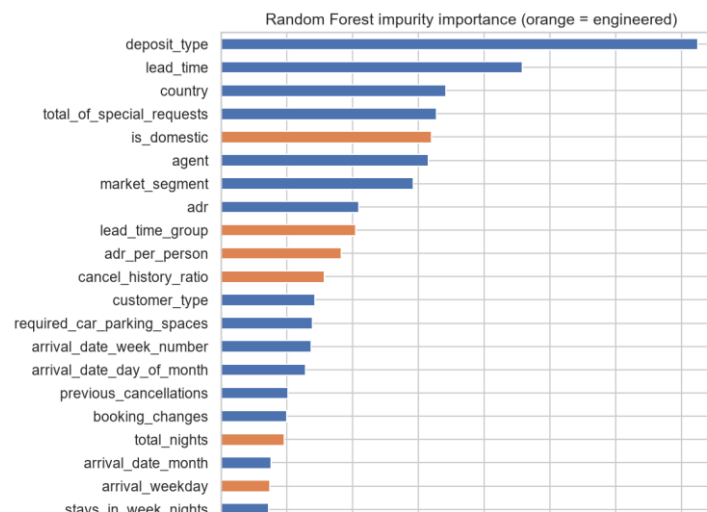
Mutual Information

Filter — สถิติล้วน



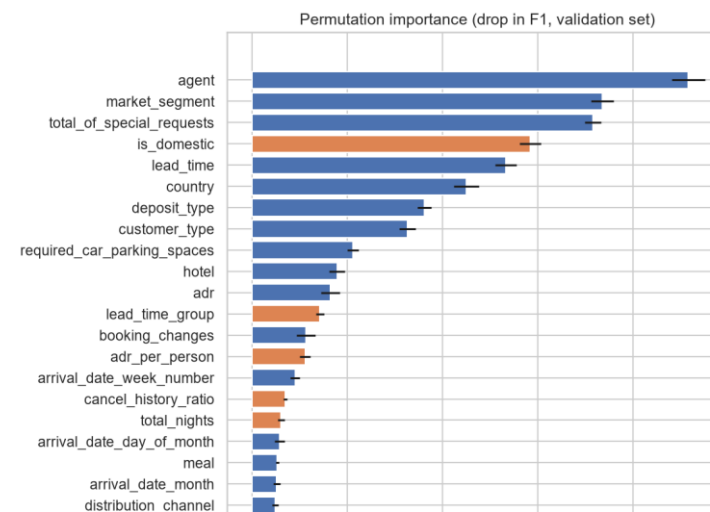
RF Importance

Embedded — จากตัวโมเดล



Permutation Importance

Model-agnostic — วัดบน validation



14 ตัว

ได้ครบ 3 โหวต

4 ตัว

ได้ 2 โหวต

18 ตัว

เก็บไว้ใช้แทน

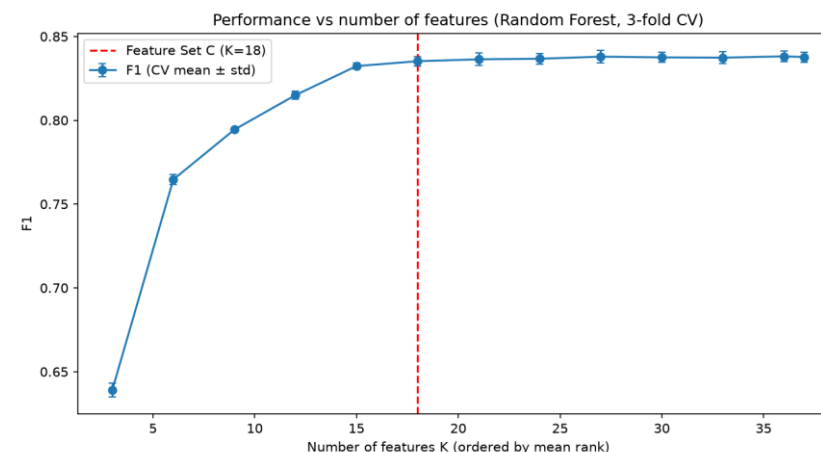
RF Importance มีอคติเข้าข้างตัวแปรที่มีค่าหลากหลาย
จึงใช้ Permutation Importance ที่วัดบน validation
set เป็นตัวถ่วง

ใช้ feature น้อยลงครึ่งหนึ่ง แต่ผลเท่าเดิมและเทรนเร็วขึ้น

ชุด Feature	จำนวน	F1 (CV)	ROC-AUC
A: ตัวแปรเดิม	28	0.8416 ± 0.0037	0.9551
B: เดิม + ที่สร้างใหม่	37	0.8427 ± 0.0028	0.9554
C: คัดเลือกแล้ว	18	0.8421 ± 0.0036	0.9546

ผลต่างระหว่าง B กับ C = **0.0006** ซึ่งน้อยกว่าค่า std (0.0028)

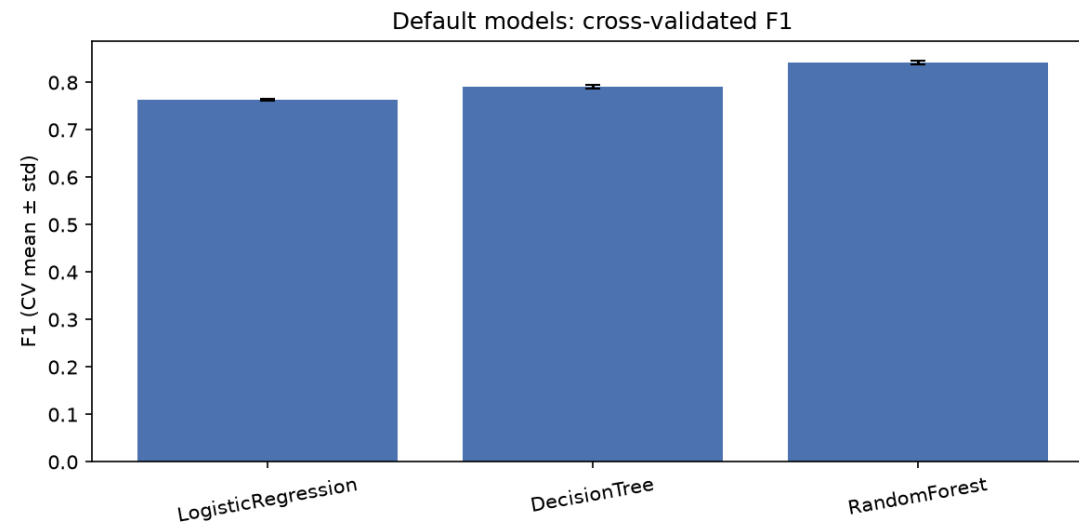
→ ไม่ต่างกันอย่างมีนัยสำคัญ แต่ C ใช้ feature ครึ่งเดียวและเทรนเร็วขึ้นเท่าตัว (8.7 vs 17.5 วินาที)



กราฟตอบคำถาม “ทำไมเลือก K = 18” — เพิ่ม feature เกินจุดนี้แล้วคะแนนไม่ขึ้นอีก

Random Forest นำตั้งแต่ก่อน Tuning (F1 = 0.842)

โมเดล (ค่า default)	F1 (CV)	ช่องว่าง Train-CV
Logistic Regression	0.763	0.001
Decision Tree	0.791	0.203
Random Forest	0.842	0.152



Decision Tree overfit ชัดเจน — ช่องว่าง train-CV ถึง 0.203

Random Forest แก้ปัญหานี้ด้วยการรวมหลายต้นเข้าด้วยกัน

ทุกโมเดลใช้ Stratified 5-Fold Cross-Validation บน train set เดียวกัน และใช้ Pipeline ชุดเดียวกัน

Accuracy หลอกตา — ทายว่า "ไม่ยกเลิก" ทุกรายการก็ได้ 62.9%

False Negative

ทำนายว่าไม่ยกเลิก → แต่ยกเลิกจริง

ห้องว่าง ขายต่อไม่ทัน = รายได้หาย

False Positive

ทำนายว่ายกเลิก → แต่ลูกค้ามาจริง

ถ้าเผื่อ **Overbooking** ไว้ = ห้องไม่พอ เสียชื่อเสียง

ทั้งสองแบบมีต้นทุนสูงพอๆกัน จึงเลือก **F1-score** เป็นตัวชี้วัดหลัก

F1-score

ตัวชี้วัดหลัก — สมดุล Precision กับ Recall

ROC-AUC

ความสามารถแยกคลาสโดยไม่ขึ้นกับ threshold

PR-AUC

เหมาะกับข้อมูลไม่สมดุลมากกว่า ROC-AUC

Confusion Matrix

เห็นจำนวน error แต่ละแบบจริง

เกณฑ์ขั้นต่ำ: การทายว่า "ไม่ยกเลิก" ทุกรายการได้ Accuracy 62.9% แต่ $F1 = 0$ คือจับคนที่จะไม่ยกเลิกไม่ได้เลยแม้แต่รายเดียว โมเดลจึงต้องชนะเกณฑ์นี้ให้ได้

Tuning ช่วยลด Overfitting มากกว่าจะเพิ่มคะแนน

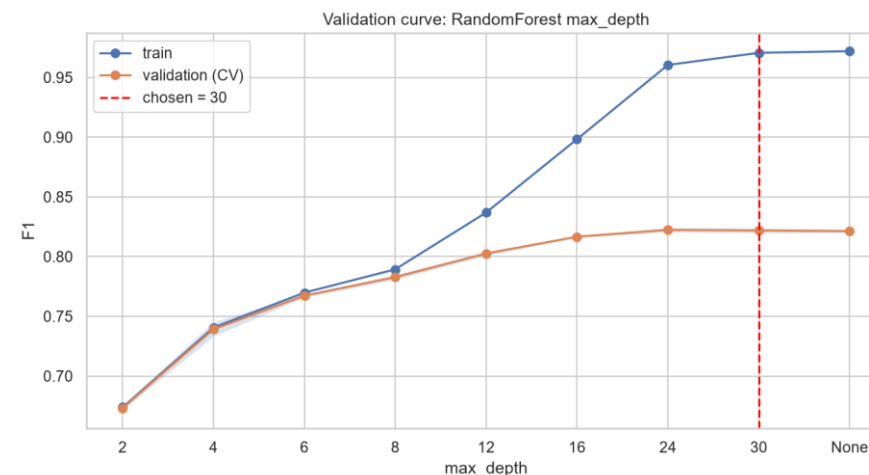
โมเดล	F1 ก่อน	F1 หลัง	ช่องว่าง Train-CV
Logistic Regression	0.7629	0.7784	0.001 → 0.002
Random Forest	0.8422	0.8419	0.152 → 0.122

F1 ของ Random Forest แทบไม่ขึ้นเลย แต่ช่องว่าง train-CV แดบลงจาก 0.152 เหลือ 0.122

→ โมเดลจำข้อมูลน้อยลงและมีเสถียรภาพมากขึ้น จึงถือว่า **tuning** ได้ผล

ค่าที่เลือกใช้:

`n_estimators=300 · max_depth=30 · min_samples_leaf=2 · max_features=log2 ·
class_weight=balanced_subsample`



Validation Curve: เส้น train พุ่งขึ้นแต่ validation นิ่ง = สัญญาณ overfitting ใช้เลือกค่า max_depth

13

ผลงานชุดทดสอบ

ปรับ threshold เป็น 0.46 เพิ่ม Recall ได้ 2.6 จุด

RandomForest on test set (t=0.46)

True label	Not canceled	13,503	1,499
	Canceled	1,246	7,594
		Not canceled	Canceled
		Predicted label	

0.8469

F1-score

0.8351

Precision

0.8590

Recall

0.9567

ROC-AUC

0.9367

PR-AUC

0.8849

Accuracy

Test set 23,842 แถว ถูกใช้เพียงครั้งเดียวตอนท้าย

การเลือก feature · เลือกโมเดล · tuning · หา threshold ทั้งหมดใช้ Cross-Validation บน train เท่านั้น

ถ้าทดสอบแบบทำนายอนาคตจริง F1 ตกจาก 0.847 เหลือ 0.725

วิธีแบ่งข้อมูล (threshold 0.46)	F1	Precision	Recall
Random split	0.8469	0.8351	0.8590
Time-based split	0.7249	0.7996	0.6630

สาเหตุ: ชุด test ที่แบ่งตามเวลามีอัตรายกเลิก 40.99% ขณะที่ train มี 36.09%
พฤติกรรมลูกค้าเปลี่ยนไปตามเวลาจริง (distribution shift)

-0.122

F1 ที่หายไปเมื่อทดสอบตามเวลา

**"การสุ่มแบ่งข้อมูลประเมินโมเดลสูงเกินจริง เรา
จึงรายงานตัวเลขทั้งสองแบบ"**

ในการใช้งานจริง โมเดลต้องทำนายอนาคตจากข้อมูลอดีต
จึงควร retrain เป็นระยะ

15

DEPLOYMENT

กรอกข้อมูลการจอง → ได้ความเสียหายทันที

เว็บแอป Streamlit — ผู้ใช้กรอกรายละเอียดการจอง ระบบคำนวณ 18 features แล้วแสดงความเสี่ยงจะเป็นที่จะยกเลิก

100%

เสี่ยงสูง

จองล่วงหน้า 300 วัน

Non Refund + Groups

33.3%

เสี่ยงปานกลาง

จองล่วงหน้า 60 วัน

Online TA

0.6%

เสี่ยงต่ำ

จองล่วงหน้า 5 วัน

Direct + ลูกค้าเก่า

ข้อจำกัดของระบบ

โมเดลบนเว็บเป็น Random Forest ฉบับย่อ 28.8 MB (F1 0.8376) แทนตัวเต็ม 356.8 MB เพราะข้อจำกัดด้านหน่วยความจำของบริการฟรี · ผลทำนายอิงข้อมูลโปรตุเกสปี 2015–2017

สรุปผล ข้อจำกัด และงานต่อไป

ตอบโจทย์หรือไม่

- ทำนายได้ F1 0.8469 บนชุดทดสอบ
- สูงกว่า baseline (F1 = 0) อย่างชัดเจน
- ระบุปัจจัยเสี่ยงหลักได้: มัดจำ · เอเจนต์ · ระยะจอล่วงหน้า

ข้อจำกัด

- ข้อมูลจากโปรดเกส ปี 2015–2017 เท่านั้น
- ทดสอบตามเวลาได้ F1 ~0.72 ต่ำกว่าผลสุ่มแบ่ง
- มีแถวซ้ำ 31,994 แถวที่เลือกไม่ลบ

งานต่อไป

- ใช้ time-based validation ตอน tuning
- ตั้ง threshold ตามต้นทุนธุรกิจจริง
- เพิ่ม SHAP อธิบายผลเป็นรายรายการ

สิ่งที่ได้เรียนรู้

ระวัง Data Leakage ตั้งแต่ก่อนแบ่งข้อมูล · เลือก metric ให้ตรงกับต้นทุนของปัญหา · ใช้ Pipeline ปกป้องกันข้อมูลรั่วระหว่าง Cross-Validation · ตัวเลขที่ดูดีเกินจริงมักมาจากวิธีประเมินที่ไม่ตรงกับการใช้งานจริง

ขอบคุณครับ / ค่ะ

Q & A

โค้ดทั้งหมด (Colab / GitHub)

github.com/Hake2549/ML

สไลด์สำรอง (เปิดเมื่อถูกถาม)

- RF Importance ก่อน-หลัง tune (Spearman 0.985)
- Learning Curve
- Error Analysis แยกตามกลุ่ม
- ช่วงค่า Hyperparameter ที่ค้นหา
- Correlation Heatmap